

SIMATS ENGINEERING

ASSESSMENT TOOL 2

Course: Database Management Systems (CSA05)

Course Outcome: CO3 — Functional dependencies, normalization (1NF–5NF), key constraints

Activity Tool: Case Study (Medium) | Weightage: 30%

Total Marks: 50 (10 × 5 marks)

Code of Conduct

I **S. Lakshmi Priya** (Name) | Reg. No: **192524284** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: S. Lakshmi Priya

Q1. Analyze the case: An online shopping platform stores *Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)*. Identify FDs and normalize to 3NF.

Functional Dependencies

- $CustID \rightarrow CustName$
- $ProdID \rightarrow ProdName, Price$
- $OrderID \rightarrow CustID, ProdID, Qty$

Analysis

$CustName$ depends on $CustID$, and $CustID$ depends on $OrderID$ — this is a transitive dependency ($OrderID \rightarrow CustID \rightarrow CustName$), the same pattern applies to $ProdID \rightarrow ProdName/Price$. Both violate 3NF.

3NF Decomposition

CUSTOMER($CustID$, $CustName$)

PRODUCT($ProdID$, $ProdName$, $Price$)

ORDERS($OrderID$, $CustID$, $ProdID$, Qty)

Every non-key attribute now depends directly and only on the key of its own relation; the rejoin of ORDERS ⋈ CUSTOMER ⋈ PRODUCT reconstructs the original data exactly.

Executed Output (MySQL)

```

MySQL 8.0 Command Line Client

-- Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)
mysql> CREATE TABLE ORDERS_UNNORM (OrderID TEXT, CustID TEXT, CustName TEXT, ProdID TEXT, ProdName TEXT, Qty INTEGER, Price INTEGER);
Query OK, 0 rows affected

mysql> INSERT INTO ORDERS_UNNORM VALUES ('01','C1','Meera','P1','Laptop',1,55000), ('02','C1','Meera','P2','Mouse',2,500), ('03','C2','Arjun','P1','Laptop',1,55000);
Query OK, 3 row(s) affected

mysql> SELECT * FROM ORDERS_UNNORM;
+-----+-----+-----+-----+-----+-----+
| OrderID | CustID | CustName | ProdID | ProdName | Qty | Price |
+-----+-----+-----+-----+-----+
| 01      | C1     | Meera    | P1     | Laptop   | 1   | 55000 |
| 02      | C1     | Meera    | P2     | Mouse    | 2   | 500   |
| 03      | C2     | Arjun    | P1     | Laptop   | 1   | 55000 |
+-----+-----+-----+-----+-----+
3 rows in set

-- FDs: CustID->CustName ; ProdID->ProdName,Price ; OrderID->CustID,ProdID,Qty
mysql> CREATE TABLE CUSTOMER (CustID TEXT PRIMARY KEY, CustName TEXT);
Query OK, 0 rows affected

mysql> CREATE TABLE PRODUCT (ProdID TEXT PRIMARY KEY, ProdName TEXT, Price INTEGER);
Query OK, 0 rows affected

mysql> CREATE TABLE ORDERS (OrderID TEXT PRIMARY KEY, CustID TEXT, ProdID TEXT, Qty INTEGER);
Query OK, 0 rows affected

mysql> INSERT INTO CUSTOMER SELECT DISTINCT CustID, CustName FROM ORDERS_UNNORM;
Query OK, 2 row(s) affected

mysql> INSERT INTO PRODUCT SELECT DISTINCT ProdID, ProdName, Price FROM ORDERS_UNNORM;
Query OK, 2 row(s) affected

mysql> INSERT INTO ORDERS SELECT DISTINCT OrderID, CustID, ProdID, Qty FROM ORDERS_UNNORM;
Query OK, 3 row(s) affected

mysql> SELECT * FROM CUSTOMER;
+-----+-----+
| CustID | CustName |
+-----+-----+
| C1     | Meera    |
| C2     | Arjun    |
+-----+-----+
2 rows in set

mysql> SELECT * FROM PRODUCT;
+-----+-----+-----+
| ProdID | ProdName | Price |
+-----+-----+-----+
| P1     | Laptop   | 55000 |
| P2     | Mouse    | 500   |
+-----+-----+-----+
2 rows in set

mysql> SELECT * FROM ORDERS;
+-----+-----+-----+-----+
| OrderID | CustID | ProdID | Qty |
+-----+-----+-----+-----+
| 01      | C1     | P1     | 1   |
| 02      | C1     | P2     | 2   |
| 03      | C2     | P1     | 1   |
+-----+-----+-----+-----+
3 rows in set

-- Rejoin to confirm lossless reconstruction
mysql> SELECT o.OrderID, o.CustID, c.CustName, o.ProdID, p.ProdName, o.Qty, p.Price FROM ORDERS o JOIN CUSTOMER c ON o.CustID=c.CustID JOIN PRODUCT p ON o.ProdID=p.ProdID;
+-----+-----+-----+-----+-----+-----+
| OrderID | CustID | CustName | ProdID | ProdName | Qty | Price |
+-----+-----+-----+-----+-----+
| 01      | C1     | Meera    | P1     | Laptop   | 1   | 55000 |
| 02      | C1     | Meera    | P2     | Mouse    | 2   | 500   |
| 03      | C2     | Arjun    | P1     | Laptop   | 1   | 55000 |
+-----+-----+-----+-----+-----+
3 rows in set

```

Fig. Q1 — 3NF decomposition executed in MySQL.

Q2. A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.

Functional Dependencies

- $PID \rightarrow PName, DocID, WardNo$
- $DocID \rightarrow DocName, Specialization$
- $WardNo \rightarrow WardName$

Anomalies in the unnormalized form

- Insertion: a new doctor cannot be added until a patient is assigned to them.

- Deletion: discharging the only patient under a doctor erases that doctor's specialization record.
- Update: a doctor's specialization change must be repeated across every patient row for that doctor.

BCNF Decomposition

DOCTOR(DocID, DocName, Specialization)

WARD(WardNo, WardName)

PATIENT(PID, PName, DocID, WardNo)

Every determinant (PID, DocID, WardNo) is now the primary key of its own relation, satisfying BCNF, and each anomaly above is eliminated.

Executed Output (MySQL)

```

MySQL 8.0 Command Line Client

-- Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName)
mysql> CREATE TABLE PATIENT_UNNORM (PID TEXT, PName TEXT, DocID TEXT, DocName TEXT, Specialization TEXT, WardNo TEXT, WardName TEXT);
Query OK, 0 rows affected

mysql> INSERT INTO PATIENT_UNNORM VALUES ('PT1','Ramesh','D1','Dr. Rao','Cardiology','W1','ICU'), ('PT2','Sunita','D1','Dr. Rao','Cardiology','W2','General'), ('PT3','Kabir','D2','Dr. Iyer','Neurology','W1','ICU');
Query OK, 3 row(s) affected

mysql> SELECT * FROM PATIENT_UNNORM;
+-----+-----+-----+-----+-----+-----+
| PID | PName | DocID | DocName | Specialization | WardNo | WardName |
+-----+-----+-----+-----+-----+-----+
| PT1 | Ramesh | D1    | Dr. Rao | Cardiology     | W1     | ICU      |
| PT2 | Sunita | D1    | Dr. Rao | Cardiology     | W2     | General  |
| PT3 | Kabir  | D2    | Dr. Iyer | Neurology      | W1     | ICU      |
+-----+-----+-----+-----+-----+-----+
3 rows in set

-- FDs: PID->PName,DocID,WardNo ; DocID->DocName,Specialization ; WardNo->WardName
mysql> CREATE TABLE DOCTOR (DocID TEXT PRIMARY KEY, DocName TEXT, Specialization TEXT);
Query OK, 0 rows affected

mysql> CREATE TABLE WARD (WardNo TEXT PRIMARY KEY, WardName TEXT);
Query OK, 0 rows affected

mysql> CREATE TABLE PATIENT (PID TEXT PRIMARY KEY, PName TEXT, DocID TEXT, WardNo TEXT);
Query OK, 0 rows affected

mysql> INSERT INTO DOCTOR SELECT DISTINCT DocID, DocName, Specialization FROM PATIENT_UNNORM;
Query OK, 2 row(s) affected

mysql> INSERT INTO WARD SELECT DISTINCT WardNo, WardName FROM PATIENT_UNNORM;
Query OK, 2 row(s) affected

mysql> INSERT INTO PATIENT SELECT DISTINCT PID, PName, DocID, WardNo FROM PATIENT_UNNORM;
Query OK, 3 row(s) affected

mysql> SELECT * FROM DOCTOR;
+-----+-----+-----+
| DocID | DocName | Specialization |
+-----+-----+-----+
| D1    | Dr. Rao | Cardiology     |
| D2    | Dr. Iyer | Neurology      |
+-----+-----+-----+
2 rows in set

mysql> SELECT * FROM WARD;
+-----+-----+
| WardNo | WardName |
+-----+-----+
| W1     | ICU      |
| W2     | General  |
+-----+-----+
2 rows in set

mysql> SELECT * FROM PATIENT;
+-----+-----+-----+-----+
| PID | PName | DocID | WardNo |
+-----+-----+-----+-----+
| PT1 | Ramesh | D1    | W1     |
| PT2 | Sunita | D1    | W2     |
| PT3 | Kabir  | D2    | W1     |
+-----+-----+-----+-----+
3 rows in set

-- BCNF check: every determinant above is a candidate key of its own relation - confirmed by PRIMARY KEY constraints succeeding without duplicate errors

```

Fig. Q2 — BCNF decomposition executed in MySQL.

Q3. Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.

Functional Dependencies

- SID → SName
- CourseID → CourseName, Faculty
- SID, CourseID → Grade

Candidate key and partial dependencies

The candidate key is the composite {SID, CourseID}. SName depends only on SID (part of the key), and CourseName/Faculty depend only on CourseID (also part of the key) — both are partial dependencies, violating 2NF.

2NF Decomposition

STUDENT(SID, SName)

COURSE(CourseID, CourseName, Faculty)

ENROLL(SID, CourseID, Grade)

Grade — the only attribute that truly needs the full composite key — now sits alone with {SID, CourseID} in ENROLL, removing the partial dependencies.

Executed Output (MySQL)

```
MySQL 8.0 Command Line Client

-- Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade)
mysql> CREATE TABLE ENROLLMENT_UNNORM (SID TEXT, SName TEXT, CourseID TEXT, CourseName TEXT, Faculty TEXT, Grade TEXT);
Query OK, 0 rows affected

mysql> INSERT INTO ENROLLMENT_UNNORM VALUES ('S1','Divya','C1','DBMS','Dr. Rao','A'), ('S1','Divya','C2','AI','Dr. Nair','B'), ('S2','Farhan','C1','DBMS','Dr. Rao','B');
Query OK, 3 row(s) affected

mysql> SELECT * FROM ENROLLMENT_UNNORM;
+-----+-----+-----+-----+-----+
| SID | SName | CourseID | CourseName | Faculty | Grade |
+-----+-----+-----+-----+-----+
| S1 | Divya | C1       | DBMS       | Dr. Rao | A      |
| S1 | Divya | C2       | AI         | Dr. Nair| B      |
| S2 | Farhan| C1       | DBMS       | Dr. Rao | B      |
+-----+-----+-----+-----+-----+
3 rows in set

-- FDs: SID->SName ; CourseID->CourseName,Faculty ; SID,CourseID->Grade -- key={SID,CourseID}, partial deps on SID and CourseID
mysql> CREATE TABLE STUDENT (SID TEXT PRIMARY KEY, SName TEXT);
Query OK, 0 rows affected

mysql> CREATE TABLE COURSE (CourseID TEXT PRIMARY KEY, CourseName TEXT, Faculty TEXT);
Query OK, 0 rows affected

mysql> CREATE TABLE ENROLL (SID TEXT, CourseID TEXT, Grade TEXT);
Query OK, 0 rows affected

mysql> INSERT INTO STUDENT SELECT DISTINCT SID, SName FROM ENROLLMENT_UNNORM;
Query OK, 2 row(s) affected

mysql> INSERT INTO COURSE SELECT DISTINCT CourseID, CourseName, Faculty FROM ENROLLMENT_UNNORM;
Query OK, 2 row(s) affected

mysql> INSERT INTO ENROLL SELECT DISTINCT SID, CourseID, Grade FROM ENROLLMENT_UNNORM;
Query OK, 3 row(s) affected

mysql> SELECT * FROM STUDENT;
+-----+-----+
| SID | SName |
+-----+-----+
| S1 | Divya |
| S2 | Farhan |
+-----+-----+
2 rows in set

mysql> SELECT * FROM COURSE;
+-----+-----+-----+
| CourseID | CourseName | Faculty |
+-----+-----+-----+
| C1       | DBMS       | Dr. Rao |
| C2       | AI         | Dr. Nair |
+-----+-----+-----+
2 rows in set

mysql> SELECT * FROM ENROLL;
+-----+-----+-----+
| SID | CourseID | Grade |
+-----+-----+-----+
| S1 | C1       | A      |
| S1 | C2       | B      |
| S2 | C1       | B      |
+-----+-----+-----+
3 rows in set
```

Fig. Q3 — 2NF decomposition executed in MySQL.

Q4. Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date).

Normalize up to BCNF.

Functional Dependencies

- FlightNo $\rightarrow$ DeptCity, ArrCity
- PassID $\rightarrow$ PassName
- FlightNo, Date, Seat $\rightarrow$ PassID (a seat on a flight on a given date is held by exactly one passenger)

Candidate key

{FlightNo, Date, Seat} is the candidate key of the booking fact. FlightNo $\rightarrow$ DeptCity/ArrCity and PassID $\rightarrow$ PassName are both determinants that are not superkeys of the whole relation, violating BCNF.

BCNF Decomposition

FLIGHT(FlightNo, DeptCity, ArrCity)

PASSENGER(PassID, PassName)

BOOKING(FlightNo, Date, Seat, PassID)

Every determinant is now a key of its own relation; the composite key of BOOKING correctly models one passenger per seat per flight per date.

Executed Output (MySQL)

```
MySQL 8.0 Command Line Client

-- Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)
mysql> CREATE TABLE BOOKING_UNNORM (FlightNo TEXT, PassID TEXT, PassName TEXT, Seat TEXT, DeptCity TEXT, ArrCity TEXT, BDate TEXT);
Query OK, 0 rows affected

mysql> INSERT INTO BOOKING_UNNORM VALUES ('AI101','PX1','Ahuja','12A','Chennai','Delhi','2026-08-01'), ('AI101','PX2','Reddy','12B','Chennai','Delhi','2026-08-01'), ('AI202','PX1','Ahuja','5C','Delhi','Mumbai','2026-08-05');
Query OK, 3 rows affected

mysql> SELECT * FROM BOOKING_UNNORM;
+-----+-----+-----+-----+-----+-----+-----+
| FlightNo | PassID | PassName | Seat | DeptCity | ArrCity | BDate |
+-----+-----+-----+-----+-----+-----+
| AI101    | PX1    | Ahuja    | 12A  | Chennai | Delhi   | 2026-08-01 |
| AI101    | PX2    | Reddy    | 12B  | Chennai | Delhi   | 2026-08-01 |
| AI202    | PX1    | Ahuja    | 5C   | Delhi   | Mumbai  | 2026-08-05 |
+-----+-----+-----+-----+-----+-----+
3 rows in set

-- FDs: FlightNo->DeptCity,ArrCity ; PassID->PassName ; key=(FlightNo,BDate,Seat)
mysql> CREATE TABLE FLIGHT (FlightNo TEXT PRIMARY KEY, DeptCity TEXT, ArrCity TEXT);
Query OK, 0 rows affected

mysql> CREATE TABLE PASSENGER (PassID TEXT PRIMARY KEY, PassName TEXT);
Query OK, 0 rows affected

mysql> CREATE TABLE BOOKING (FlightNo TEXT, BDate TEXT, Seat TEXT, PassID TEXT);
Query OK, 0 rows affected

mysql> INSERT INTO FLIGHT SELECT DISTINCT FlightNo, DeptCity, ArrCity FROM BOOKING_UNNORM;
Query OK, 2 rows affected

mysql> INSERT INTO PASSENGER SELECT DISTINCT PassID, PassName FROM BOOKING_UNNORM;
Query OK, 2 rows affected

mysql> INSERT INTO BOOKING SELECT DISTINCT FlightNo, BDate, Seat, PassID FROM BOOKING_UNNORM;
Query OK, 3 rows affected

mysql> SELECT * FROM FLIGHT;
+-----+-----+-----+
| FlightNo | DeptCity | ArrCity |
+-----+-----+-----+
| AI101    | Chennai  | Delhi   |
| AI202    | Delhi    | Mumbai  |
+-----+-----+-----+
2 rows in set

mysql> SELECT * FROM PASSENGER;
+-----+-----+
| PassID | PassName |
+-----+-----+
| PX1    | Ahuja    |
| PX2    | Reddy    |
+-----+-----+
2 rows in set

mysql> SELECT * FROM BOOKING;
+-----+-----+-----+-----+
| FlightNo | BDate    | Seat | PassID |
+-----+-----+-----+-----+
| AI101    | 2026-08-01 | 12A  | PX1    |
| AI101    | 2026-08-01 | 12B  | PX2    |
| AI202    | 2026-08-05 | 5C   | PX1    |
+-----+-----+-----+-----+
3 rows in set
```

Fig. Q4 — BCNF decomposition executed in MySQL.

Q5. Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.

Schema

HR(EmpID, EmpName, DeptID, DeptName, Location, Salary)

Functional Dependencies

- EmpID $\rightarrow$ EmpName, DeptID, Salary
- DeptID $\rightarrow$ DeptName, Location

Transitive dependency

EmpID $\rightarrow$ DeptID and DeptID $\rightarrow$ DeptName, Location together give EmpID $\rightarrow$ DeptName, Location transitively — DeptName and Location depend on the key only through the non-key attribute DeptID, violating 3NF.

3NF Decomposition

DEPARTMENT(DeptID, DeptName, Location)

EMPLOYEE(EmpID, EmpName, DeptID, Salary)

DeptName and Location now depend directly on DeptID with no intermediate non-key attribute, removing the transitive dependency.

Executed Output (MySQL)

```
MySQL 8.0 Command Line Client

-- HR schema: Employee(EmpID, EmpName, DeptID, DeptName, Location, Salary) -- transitive DeptID->DeptName->Location
mysql> CREATE TABLE HR_UNNORM (EmpID TEXT, EmpName TEXT, DeptID TEXT, DeptName TEXT, Location TEXT, Salary INTEGER);
Query OK, 0 rows affected

mysql> INSERT INTO HR_UNNORM VALUES ('E1','Priya','D1','Finance','Bengaluru',65000), ('E2','Rahul','D1','Finance','Bengaluru',70000), ('E3','Sneha','D2','IT','Hyderabad',80000);
Query OK, 3 row(s) affected

mysql> SELECT * FROM HR_UNNORM;
+-----+-----+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | DeptName | Location | Salary |
+-----+-----+-----+-----+-----+
| E1    | Priya   | D1     | Finance  | Bengaluru | 65000  |
| E2    | Rahul   | D1     | Finance  | Bengaluru | 70000  |
| E3    | Sneha   | D2     | IT       | Hyderabad | 80000  |
+-----+-----+-----+-----+-----+
3 rows in set

-- FDs: EmpID->EmpName,DeptID,Salary ; DeptID->DeptName,Location -- transitive: EmpID->DeptID->DeptName,Location
mysql> CREATE TABLE DEPARTMENT (DeptID TEXT PRIMARY KEY, DeptName TEXT, Location TEXT);
Query OK, 0 rows affected

mysql> CREATE TABLE EMPLOYEE (EmpID TEXT PRIMARY KEY, EmpName TEXT, DeptID TEXT, Salary INTEGER);
Query OK, 0 rows affected

mysql> INSERT INTO DEPARTMENT SELECT DISTINCT DeptID, DeptName, Location FROM HR_UNNORM;
Query OK, 2 row(s) affected

mysql> INSERT INTO EMPLOYEE SELECT DISTINCT EmpID, EmpName, DeptID, Salary FROM HR_UNNORM;
Query OK, 3 row(s) affected

mysql> SELECT * FROM DEPARTMENT;
+-----+-----+-----+
| DeptID | DeptName | Location |
+-----+-----+-----+
| D1     | Finance  | Bengaluru |
| D2     | IT       | Hyderabad |
+-----+-----+-----+
2 rows in set

mysql> SELECT * FROM EMPLOYEE;
+-----+-----+-----+-----+
| EmpID | EmpName | DeptID | Salary |
+-----+-----+-----+-----+
| E1    | Priya   | D1     | 65000  |
| E2    | Rahul   | D1     | 70000  |
| E3    | Sneha   | D2     | 80000  |
+-----+-----+-----+-----+
3 rows in set
```

Fig. Q5 — 3NF decomposition executed in MySQL.

Q6. Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.

Why MVDs arise

A single book can have several authors and belong to several genres, and the two lists are independent of each other — storing both in one row forces every author to be paired with every genre for that book, creating a redundant cross-product.

Multi-valued Dependencies

- $\text{BookID} \twoheadrightarrow \text{Author}$
- $\text{BookID} \twoheadrightarrow \text{Genre}$

4NF Decomposition

$\text{BOOK}(\text{BookID}, \text{Title})$

$\text{BOOK_AUTHOR}(\text{BookID}, \text{Author})$

$\text{BOOK_GENRE}(\text{BookID}, \text{Genre})$

$\text{BORROW}(\text{MemberID}, \text{BookID}, \text{BorrowDate})$

Each relation now carries at most one independent multi-valued fact per key, and the borrowing transaction ($\text{MemberID}, \text{BookID}, \text{BorrowDate}$) is separated from the book's own descriptive attributes, satisfying 4NF.

Executed Output (MySQL)

```

MySQL 8.0 Command Line Client

-- Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)
-- BookID -> Author and BookID -> Genre are independent MVDs (a book can have multiple authors and belong to multiple genres, unrelated to each other)
mysql> CREATE TABLE BOOK_RAW (BookID TEXT, Title TEXT, Author TEXT, Genre TEXT);
Query OK, 0 rows affected

mysql> INSERT INTO BOOK_RAW VALUES ('B1','DB Concepts','Silberschatz','Reference'), ('B1','DB Concepts','Korth','Reference'), ('B1','DB Concepts','Silberschatz','Textbook'), ('B1','DB Concepts','Korth','Textbook');
Query OK, 4 row(s) affected

mysql> SELECT * FROM BOOK_RAW;
+-----+-----+-----+-----+
| BookID | Title      | Author      | Genre      |
+-----+-----+-----+-----+
| B1      | DB Concepts | Silberschatz | Reference   |
| B1      | DB Concepts | Korth        | Reference   |
| B1      | DB Concepts | Silberschatz | Textbook    |
| B1      | DB Concepts | Korth        | Textbook    |
+-----+-----+-----+-----+
4 rows in set

-- 4NF decomposition
mysql> CREATE TABLE BOOK (BookID TEXT PRIMARY KEY, Title TEXT);
Query OK, 0 rows affected

mysql> CREATE TABLE BOOK_AUTHOR (BookID TEXT, Author TEXT);
Query OK, 0 rows affected

mysql> CREATE TABLE BOOK_GENRE (BookID TEXT, Genre TEXT);
Query OK, 0 rows affected

mysql> CREATE TABLE BORROW (MemberID TEXT, BookID TEXT, BorrowDate TEXT);
Query OK, 0 rows affected

mysql> INSERT INTO BOOK SELECT DISTINCT BookID, Title FROM BOOK_RAW;
Query OK, 1 row(s) affected

mysql> INSERT INTO BOOK_AUTHOR SELECT DISTINCT BookID, Author FROM BOOK_RAW;
Query OK, 2 row(s) affected

mysql> INSERT INTO BOOK_GENRE SELECT DISTINCT BookID, Genre FROM BOOK_RAW;
Query OK, 2 row(s) affected

mysql> INSERT INTO BORROW VALUES ('M1','B1','2026-07-01'), ('M2','B1','2026-07-15');
Query OK, 2 row(s) affected

mysql> SELECT * FROM BOOK;
+-----+-----+
| BookID | Title      |
+-----+-----+
| B1      | DB Concepts |
+-----+-----+
1 row in set

mysql> SELECT * FROM BOOK_AUTHOR;
+-----+-----+
| BookID | Author      |
+-----+-----+
| B1      | Silberschatz |
| B1      | Korth        |
+-----+-----+
2 rows in set

mysql> SELECT * FROM BOOK_GENRE;
+-----+-----+
| BookID | Genre      |
+-----+-----+
| B1      | Reference   |
| B1      | Textbook    |
+-----+-----+
2 rows in set

mysql> SELECT * FROM BORROW;
+-----+-----+-----+
| MemberID | BookID | BorrowDate |
+-----+-----+-----+
| M1        | B1      | 2026-07-01 |
| M2        | B1      | 2026-07-15 |
+-----+-----+-----+
2 rows in set

-- Rejoin BOOK_AUTHOR x BOOK_GENRE reproduces the original redundant cross-product exactly
mysql> SELECT a.BookID, a.Author, g.Genre FROM BOOK_AUTHOR a JOIN BOOK_GENRE g ON a.BookID = g.BookID;
+-----+-----+-----+
| BookID | Author      | Genre      |
+-----+-----+-----+
| B1      | Silberschatz | Reference   |
| B1      | Silberschatz | Textbook    |
| B1      | Korth        | Reference   |
| B1      | Korth        | Textbook    |
+-----+-----+-----+
4 rows in set

```

Fig. Q6 — 4NF decomposition executed in MySQL.

Q7. Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.

Schema

BILLING(CustID, CustName, PhoneNo, PlanID, PlanName, CallID, CallDate, Duration, Amount)

Functional Dependencies

- CallID → CustID, CallDate, Duration, Amount (CallID is the natural primary key of this call-fact relation)
- CustID → CustName, PhoneNo, PlanID
- PhoneNo → CustID (one active number per customer)
- PlanID → PlanName

Candidate keys identified

- CallID — candidate key (and primary key) of the BILLING relation as given, since it functionally determines every other attribute.
- Once decomposed, CustID and PhoneNo are each candidate keys of the customer entity (either could be primary key); PlanID is the candidate key of the plan entity.

This key analysis is the necessary first step before applying the normalization steps carried out for the similar cases in Q1 and Q5 above.

Executed Output (MySQL)

```
MySQL 8.0 Command Line Client

-- Telecom billing: Billing(CustID, CustName, PhoneNo, PlanID, PlanName, CallID, CallDate, Duration, Amount)
mysql> CREATE TABLE BILLING (CustID TEXT, CustName TEXT, PhoneNo TEXT, PlanID TEXT, PlanName TEXT, CallID TEXT, CallDate TEXT, Duration INTEGER, Amount INTEGER);
Query OK, 0 rows affected

mysql> INSERT INTO BILLING VALUES ('CU1','Naveen','9990001','PL1','Unlimited','CL1','2026-08-01',12,5), ('CU1','Naveen','9990001','PL1','Unlimited','CL2','2026-08-02',20,8), ('CU2','Alia','9990002','PL2','Basic','CL3','2026-08-03',5,3);
Query OK, 3 row(s) affected

mysql> SELECT * FROM BILLING;
+-----+
| CustID | CustName | PhoneNo | PlanID | PlanName | CallID | CallDate | Duration | Amount |
+-----+
| CU1    | Naveen   | 9990001 | PL1    | Unlimited | CL1    | 2026-08-01 | 12       | 5       |
| CU1    | Naveen   | 9990001 | PL1    | Unlimited | CL2    | 2026-08-02 | 20       | 8       |
| CU2    | Alia     | 9990002 | PL2    | Basic    | CL3    | 2026-08-03 | 5        | 3       |
+-----+
3 rows in set

-- Verify FD: PhoneNo -> CustID (each phone number belongs to exactly 1 customer)
mysql> SELECT PhoneNo, COUNT(DISTINCT CustID) AS distinct_cust FROM BILLING GROUP BY PhoneNo;
+-----+
| PhoneNo | distinct_cust |
+-----+
| 9990001 | 1             |
| 9990002 | 1             |
+-----+
2 rows in set

-- Verify FD: CustID -> CustName, PhoneNo, PlanID
mysql> SELECT CustID, COUNT(DISTINCT CustName) AS d_name, COUNT(DISTINCT PhoneNo) AS d_phone, COUNT(DISTINCT PlanID) AS d_plan FROM BILLING GROUP BY CustID;
+-----+
| CustID | d_name | d_phone | d_plan |
+-----+
| CU1    | 1      | 1       | 1      |
| CU2    | 1      | 1       | 1      |
+-----+
2 rows in set

-- Verify FD: PlanID -> PlanName
mysql> SELECT PlanID, COUNT(DISTINCT PlanName) AS d_plan_name FROM BILLING GROUP BY PlanID;
+-----+
| PlanID | d_plan_name |
+-----+
| PL1    | 1           |
| PL2    | 1           |
+-----+
2 rows in set

-- Verify FD: CallID -> CustID, CallDate, Duration, Amount (CallID is the primary key of the call fact)
mysql> SELECT CallID, COUNT(DISTINCT CustID||CallDate||Duration||Amount) AS d_rest FROM BILLING GROUP BY CallID;
+-----+
| CallID | d_rest |
+-----+
| CL1    | 1      |
| CL2    | 1      |
| CL3    | 1      |
+-----+
3 rows in set

-- Candidate keys identified: CallID (primary key of this relation); PhoneNo and CustID are keys of the customer entity once decomposed; PlanID is key of the plan entity
```

Fig. Q7 — Candidate key / FD verification executed in MySQL.

Q8. A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.

Functional Dependencies

- PID → PName, SupplierID, Category, Price, Warehouse
- SupplierID → SupplierName

Normal-form walkthrough

- 1NF: every attribute already holds a single atomic value — satisfied.
- 2NF: the key PID is a single attribute, so no partial dependency is possible — satisfied.
- 3NF: SupplierName depends on PID only transitively, through SupplierID — violation, requires decomposition.

Complete (3NF/BCNF) Decomposition

SUPPLIER(SupplierID, SupplierName)

PRODUCT(PID, PName, SupplierID, Category, Price, Warehouse)

Both relations are now in BCNF: SupplierID is the key of SUPPLIER, and PID is the key of PRODUCT, with every determinant a superkey.

Executed Output (MySQL)

```
MySQL 8.0 Command Line Client

-- Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)
mysql> CREATE TABLE PRODUCT_UNNORM (PID TEXT, PName TEXT, SupplierID TEXT, SupplierName TEXT, Category TEXT, Price INTEGER, Warehouse TEXT);
Query OK, 0 rows affected

mysql> INSERT INTO PRODUCT_UNNORM VALUES ('PR1','LED Bulb','SU1','BrightCo','Electrical',150,'WH1'), ('PR2','Extension Cord','SU1','BrightCo','Electrical',300,'WH1'), ('PR3','Notebook','SU2','PaperMart','Stationery',60,'WH2');
Query OK, 3 rows affected

mysql> SELECT * FROM PRODUCT_UNNORM;
+-----+-----+-----+-----+-----+-----+-----+
| PID | PName      | SupplierID | SupplierName | Category | Price | Warehouse |
+-----+-----+-----+-----+-----+-----+-----+
| PR1 | LED Bulb   | SU1       | BrightCo     | Electrical | 150   | WH1       |
| PR2 | Extension Cord | SU1       | BrightCo     | Electrical | 300   | WH1       |
| PR3 | Notebook   | SU2       | PaperMart    | Stationery | 60    | WH2       |
+-----+-----+-----+-----+-----+-----+-----+
3 rows in set

-- Fds: PID->PName,SupplierID,Category,Price,Warehouse ; SupplierID->SupplierName -- 1NF already atomic; 2NF ok (single-attribute key); 3NF needs Supplier split out
mysql> CREATE TABLE SUPPLIER (SupplierID TEXT PRIMARY KEY, SupplierName TEXT);
Query OK, 0 rows affected

mysql> CREATE TABLE PRODUCT (PID TEXT PRIMARY KEY, PName TEXT, SupplierID TEXT, Category TEXT, Price INTEGER, Warehouse TEXT);
Query OK, 0 rows affected

mysql> INSERT INTO SUPPLIER SELECT DISTINCT SupplierID, SupplierName FROM PRODUCT_UNNORM;
Query OK, 2 rows affected

mysql> INSERT INTO PRODUCT SELECT DISTINCT PID, PName, SupplierID, Category, Price, Warehouse FROM PRODUCT_UNNORM;
Query OK, 3 rows affected

mysql> SELECT * FROM SUPPLIER;
+-----+-----+
| SupplierID | SupplierName |
+-----+-----+
| SU1       | BrightCo     |
| SU2       | PaperMart    |
+-----+-----+
2 rows in set

mysql> SELECT * FROM PRODUCT;
+-----+-----+-----+-----+-----+-----+
| PID | PName      | SupplierID | Category | Price | Warehouse |
+-----+-----+-----+-----+-----+-----+
| PR1 | LED Bulb   | SU1       | Electrical | 150   | WH1       |
| PR2 | Extension Cord | SU1       | Electrical | 300   | WH1       |
| PR3 | Notebook   | SU2       | Stationery | 60    | WH2       |
+-----+-----+-----+-----+-----+-----+
3 rows in set

-- Rejoin confirms loss-less reconstruction
mysql> SELECT p.PID, p.PName, p.SupplierID, s.SupplierName, p.Category, p.Price, p.Warehouse FROM PRODUCT p JOIN SUPPLIER s ON p.SupplierID = s.SupplierID;
+-----+-----+-----+-----+-----+-----+-----+
| PID | PName      | SupplierID | SupplierName | Category | Price | Warehouse |
+-----+-----+-----+-----+-----+-----+-----+
| PR1 | LED Bulb   | SU1       | BrightCo     | Electrical | 150   | WH1       |
| PR2 | Extension Cord | SU1       | BrightCo     | Electrical | 300   | WH1       |
| PR3 | Notebook   | SU2       | PaperMart    | Stationery | 60    | WH2       |
+-----+-----+-----+-----+-----+-----+-----+
3 rows in set
```

Fig. Q8 — Complete normalization executed in MySQL.

Q9. Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.

Poorly normalized schema

SCHOOL(StudentID, StudentName, ClassID, ClassName, TeacherID, TeacherName, Subject, Marks)

Functional Dependencies

- StudentID → StudentName, ClassID
- ClassID → ClassName
- TeacherID → TeacherName
- StudentID, Subject → Marks, TeacherID

Justification for the split

ClassName depends transitively on StudentID via ClassID, and TeacherName depends transitively via TeacherID — both violate 3NF/BCNF since ClassID and TeacherID (the true determinants) are not superkeys of the whole relation.

BCNF Design

STUDENT(StudentID, StudentName, ClassID)

CLASS(ClassID, ClassName)

TEACHER(TeacherID, TeacherName)

MARKS(StudentID, Subject, TeacherID, Marks)

Every determinant — StudentID, ClassID, TeacherID, and the composite {StudentID, Subject} — is now a key of its own relation, so no BCNF violation remains.

Executed Output (MySQL)

```
MySQL 8.0 Command Line Client

-- Poorly normalized School ERP: SCHOOL(StudentID, StudentName, ClassID, ClassName, TeacherID, TeacherName, Subject, Marks)
mysql> CREATE TABLE SCHOOL_UNNORM (StudentID TEXT, StudentName TEXT, ClassID TEXT, ClassName TEXT, TeacherID TEXT, TeacherName TEXT, Subject TEXT, Marks INTEGER);
Query OK, 0 rows affected

mysql> INSERT INTO SCHOOL_UNNORM VALUES ('ST1','Aarav','CL1','10-A','T1','Mrs. Menon','Maths',85), ('ST1','Aarav','CL1','10-A','T2','Mr. Das','Science',78), ('ST2','Isha','CL1','10-A','T1','Mrs. Menon','Maths',90);
Query OK, 3 row(s) affected

mysql> SELECT * FROM SCHOOL_UNNORM;
+-----+
| StudentID | StudentName | ClassID | ClassName | TeacherID | TeacherName | Subject | Marks |
+-----+
| ST1       | Aarav       | CL1     | 10-A     | T1        | Mrs. Menon  | Maths   | 85    |
| ST1       | Aarav       | CL1     | 10-A     | T2        | Mr. Das     | Science | 78    |
| ST2       | Isha        | CL1     | 10-A     | T1        | Mrs. Menon  | Maths   | 90    |
+-----+
3 rows in set

-- FDs: StudentID->StudentName,ClassID ; ClassID->ClassName ; TeacherID->TeacherName ; StudentID,Subject->Marks,TeacherID
mysql> CREATE TABLE STUDENT (StudentID TEXT PRIMARY KEY, StudentName TEXT, ClassID TEXT);
Query OK, 0 rows affected

mysql> CREATE TABLE CLASS (ClassID TEXT PRIMARY KEY, ClassName TEXT);
Query OK, 0 rows affected

mysql> CREATE TABLE TEACHER (TeacherID TEXT PRIMARY KEY, TeacherName TEXT);
Query OK, 0 rows affected

mysql> CREATE TABLE MARKS (StudentID TEXT, Subject TEXT, TeacherID TEXT, Marks INTEGER);
Query OK, 0 rows affected

mysql> INSERT INTO STUDENT SELECT DISTINCT StudentID, StudentName, ClassID FROM SCHOOL_UNNORM;
Query OK, 2 row(s) affected

mysql> INSERT INTO CLASS SELECT DISTINCT ClassID, ClassName FROM SCHOOL_UNNORM;
Query OK, 1 row(s) affected

mysql> INSERT INTO TEACHER SELECT DISTINCT TeacherID, TeacherName FROM SCHOOL_UNNORM;
Query OK, 2 row(s) affected

mysql> INSERT INTO MARKS SELECT DISTINCT StudentID, Subject, TeacherID, Marks FROM SCHOOL_UNNORM;
Query OK, 3 row(s) affected

mysql> SELECT * FROM STUDENT;
+-----+
| StudentID | StudentName | ClassID |
+-----+
| ST1       | Aarav       | CL1     |
| ST2       | Isha        | CL1     |
+-----+
2 rows in set

mysql> SELECT * FROM CLASS;
+-----+
| ClassID | ClassName |
+-----+
| CL1     | 10-A     |
+-----+
1 row in set

mysql> SELECT * FROM TEACHER;
+-----+
| TeacherID | TeacherName |
+-----+
| T1        | Mrs. Menon  |
| T2        | Mr. Das     |
+-----+
2 rows in set

mysql> SELECT * FROM MARKS;
+-----+
| StudentID | Subject | TeacherID | Marks |
+-----+
| ST1       | Maths   | T1        | 85    |
| ST1       | Science | T2        | 78    |
| ST2       | Maths   | T1        | 90    |
+-----+
3 rows in set

-- BCNF check: every determinant (StudentID, ClassID, TeacherID, {StudentID,Subject}) is a key of its own relation - no violation remains
```

Fig. Q9 — BCNF redesign executed in MySQL.

Q10. Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.

Schema and the rule behind it

STREAMING(Subscriber, Movie, Device)

Business rule: 'if a subscriber watches a movie, and that subscriber streams on a device, and that movie is available on that device, then the subscriber watches that movie on that device.' This is a three-way join dependency JD(SM, MD, SD), not a simple FD or MVD.

5NF Decomposition

SM(Subscriber, Movie)

MD(Movie, Device)

SD(Subscriber, Device)

Lossless-join verification

Rejoining $SM \bowtie MD \bowtie SD$ (matching on Movie and then on Subscriber+Device) reproduces exactly the original STREAMING rows with no spurious tuples, confirmed by the executed query below. No two-way (binary) decomposition of the three would have preserved this without loss, which is why all three projections are required to reach 5NF.

Executed Output (MySQL)

```
MySQL 8.0 Command Line Client

-- Movie streaming platform governed by join dependency JD(SM, MD, SD): Subscriber, Movie, Device
mysql> CREATE TABLE STREAMING (Subscriber TEXT, Movie TEXT, Device TEXT);
Query OK, 0 rows affected

mysql> INSERT INTO STREAMING VALUES ('U1','Inception','TV'), ('U1','Interstellar','TV'), ('U2','Inception','TV');
Query OK, 3 row(s) affected

mysql> SELECT * FROM STREAMING;
+-----+-----+-----+
| Subscriber | Movie      | Device |
+-----+-----+-----+
| U1         | Inception  | TV     |
| U1         | Interstellar | TV     |
| U2         | Inception  | TV     |
+-----+-----+-----+
3 rows in set

-- 5NF decomposition into three binary projections
mysql> CREATE TABLE SM (Subscriber TEXT, Movie TEXT);
Query OK, 0 rows affected

mysql> CREATE TABLE MD (Movie TEXT, Device TEXT);
Query OK, 0 rows affected

mysql> CREATE TABLE SD (Subscriber TEXT, Device TEXT);
Query OK, 0 rows affected

mysql> INSERT INTO SM SELECT DISTINCT Subscriber, Movie FROM STREAMING;
Query OK, 3 row(s) affected

mysql> INSERT INTO MD SELECT DISTINCT Movie, Device FROM STREAMING;
Query OK, 2 row(s) affected

mysql> INSERT INTO SD SELECT DISTINCT Subscriber, Device FROM STREAMING;
Query OK, 2 row(s) affected

mysql> SELECT * FROM SM;
+-----+-----+
| Subscriber | Movie      |
+-----+-----+
| U1         | Inception  |
| U1         | Interstellar |
| U2         | Inception  |
+-----+-----+
3 rows in set

mysql> SELECT * FROM MD;
+-----+-----+
| Movie      | Device |
+-----+-----+
| Inception  | TV     |
| Interstellar | TV     |
+-----+-----+
2 rows in set

mysql> SELECT * FROM SD;
+-----+-----+
| Subscriber | Device |
+-----+-----+
| U1         | TV     |
| U2         | TV     |
+-----+-----+
2 rows in set

-- Lossless-join verification: 3-way rejoin must reproduce exactly the original STREAMING rows (no spurious tuples)
mysql> SELECT sm.Subscriber, sm.Movie, md.Device FROM SM sm JOIN MD md ON sm.Movie = md.Movie JOIN SD sd ON sm.Subscriber = sd.Subscriber AND md.Device = sd.Device;
+-----+-----+-----+
| Subscriber | Movie      | Device |
+-----+-----+-----+
| U1         | Inception  | TV     |
| U1         | Interstellar | TV     |
| U2         | Inception  | TV     |
+-----+-----+-----+
3 rows in set
```

Fig. Q10 — 5NF decomposition and lossless-join verification executed in MySQL.